1(a) Consider a system running ten I/O-bound tasks and one CPU-bound task. Assume that the I/O bound tasks issue an I/O operation once for every millisecond of CPU computing and that each I/O operation takes 10 milliseconds to complete. Also assume that the context-switching overhead is 0.1 millisecond and that all processes are long-running tasks. Describe the CPU utilization for a round-robin scheduler when :

i) **The quantum time is 1 millisecond,**
ii) **The quantum time is 10 milliseconds.**

CPU Utilization:

- In each **1 millisecond quantum**, there is a **0.1 millisecond overhead** for context switching, so the CPU is actively working for **0.9 milliseconds**.
- Therefore, the **CPU utilization** in this case is:

$$\text{CPU Utilization} = \frac{0.9 \text{ milliseconds of work}}{1 \text{ millisecond total}} = 90\%$$

The **CPU utilization** is:

$$\text{CPU Utilization} \approx \frac{9.9 \text{ milliseconds of work}}{10 \text{ milliseconds total}} = 99\%$$

1(b) State the reasons why a parent may terminate the execution of one of its children. Why is it important to choose a good process mix of I/O-bound and CPU-bound processes in case of long term scheduler?

Parent may terminate the execution of children processes using the

abort() system call. Some reasons for doing so:

1. Child has exceeded allocated resources

2. Task assigned to child is no longer required

3. The parent is exiting and the operating systems does not allow a

   child to continue if its parent terminates

1(c) What do you mean by inter-process communication(IPC) ? Explain the two fundamental models of inter-process communication with sketches.

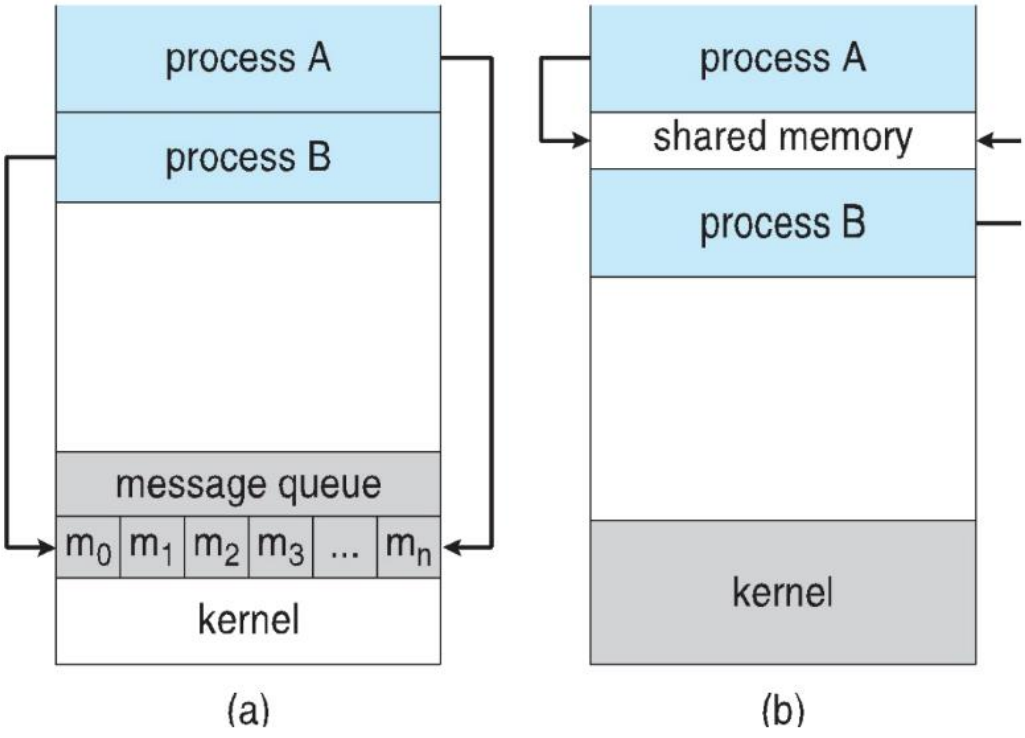
Inter-process Communication (IPC) refers to the mechanisms provided by an operating system that allow processes to communicate and coordinate their actions when they are cooperating (i.e., they share resources or data).

Reasons for cooperating processes:

- Information sharing
- Computation speedup
- Modularity
- Convenience

Communications Models

(a) Message passing. (b) shared memory.

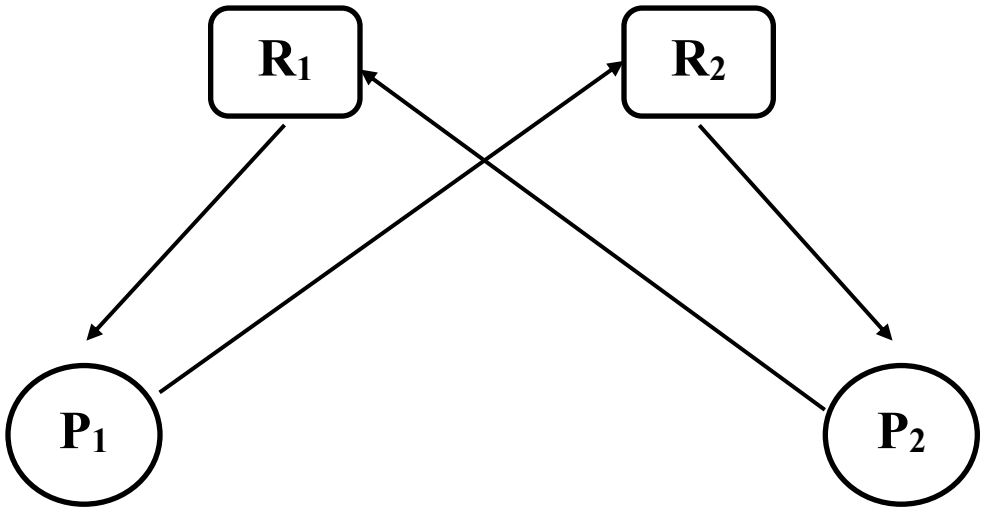


2(a) What is a deadlock? What are the necessary conditions for a deadlock to occur ?

Deadlock occurs in a system when a set of processes are permanently blocked, each waiting for a resource that another process in the set holds. For deadlock to occur, the following four conditions must hold simultaneously:

Conditions for Deadlock:

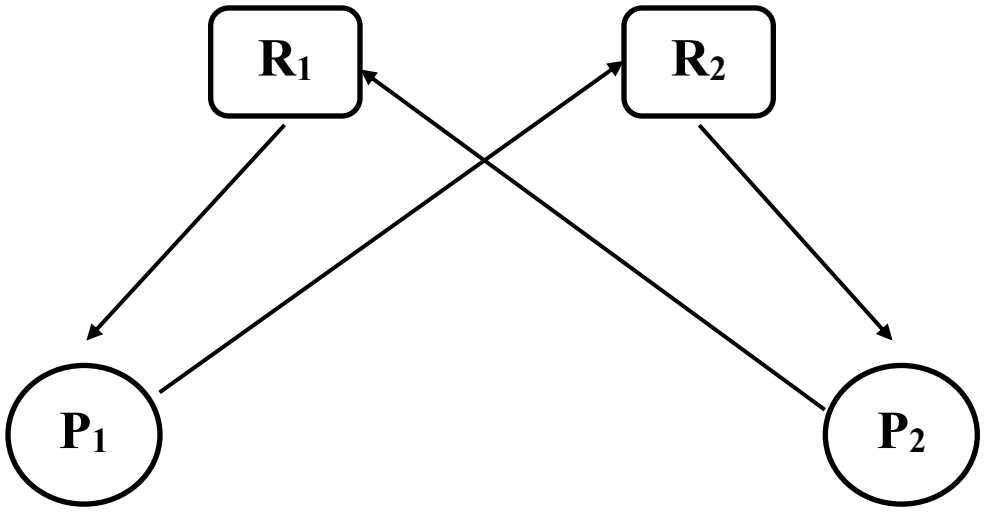
1. **Mutual Exclusion**:
    - At least one resource must be held in a **non-shareable** mode, meaning only one process can use the resource at a time.
    - If another process requests that resource, it must wait until the resource is released.
2. **Hold and Wait**:
    - A process must be **holding at least one resource** and simultaneously waiting to acquire additional resources that are currently being held by other processes.
3. **No Preemption**:
    - Resources cannot be forcibly taken away from a process. A resource can only be **released voluntarily** by the process holding it, once the process completes its task.
4. **Circular Wait**:
    - There must exist a set of processes {P1, P2, ..., Pn} such that:
        - Process P1 is waiting for a resource held by P2,
        - Process P2 is waiting for a resource held by P3,
        - ...
        - Process Pn is waiting for a resource held by P1.
    - This creates a circular chain or list where each process waits for the next process in the chain to release a resource.



In this situation:

- **Mutual Exclusion:** Both R1 and R2 can only be held by one process at a time.
- **Hold and Wait:** P1 holds R1 and waits for R2, and P2 holds R2 and waits for R1.
- **No Preemption:** Neither P1 nor P2 can be forced to release its resources.
- **Circular Wait**: P1 is waiting for P2, and P2 is waiting for P1—creating a circular dependency.

2(b) “A cycle in the graph is a necessary but not a sufficient condition for the existence of deadlock” -explain the line.



In this situation:

- **Mutual Exclusion:** Both R1 and R2 can only be held by one process at a time.
- **Hold and Wait:** P1 holds R1 and waits for R2, and P2 holds R2 and waits for R1.
- **No Preemption:** Neither P1 nor P2 can be forced to release its resources.
- **Circular Wait**: P1 is waiting for P2, and P2 is waiting for P1—creating a circular dependency.

deadlock" – explain the statement.

Q.2(c) Consider a system with the following current resource allocation state: 15

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	1	1	2	4	3	3	2	1	0
P_1	2	1	2	3	2	2			
P_2	4	0	1	9	0	2			
P_3	0	2	0	7	5	3			
P_4	1	1	2	11	2	3			

Table-2(c)

There are five processes: P_0, P_1, P_2, P_3, P_4 and three resource types: A, B, and C. For each process, the current allocation and the maximum required allocation are given by the *Allocation* and *MAX* matrices. The current available resources are given by the *Available* vector.

Answer the following questions:

i) Determine the total amount of resources of each type.

ii) What is the content of the matrix *Need*?

iii) Determine if the system is "safe" using the "Banker's Safety Algorithm".

iv) What, if anything, does this problem demonstrate about the relationship between the "safeness" of an allocation state and deadlock itself?

2(d) Why do we need to synchronize processes?

3(a) What is the meaning of the term busy waiting? What other kinds of waiting are there in an operating system ? Can busy waiting be avoided altogether? Explain your answer .

Busy Waiting and How to Avoid It

- **Busy waiting** is when a process constantly checks (or "spins") to see if a resource is available, which wastes CPU time.
- To avoid this, **sleep()** is used. The process goes to sleep while waiting for the resource and gets woken up when it's available.

```
typedef struct {
        int value;              // Semaphore value
        struct process *list;   // List of waiting processes
} semaphore;
wait(semaphore *S) {
        S->value--;  // Decrease the semaphore value

        if (S->value < 0) {  // If no resources are available
        add this process to S->list;
                // Add the current process to the  wait list
        sleep();  // Put the process to sleep
}
        }
signal(semaphore *S) {
        S->value++;  // Increase the semaphore value
        if (S->value <= 0) {  // If there are waiting processes
        remove a process P from S->list;
                        // Wake up a process from the list
```

3(b) **"Five philosophers are sitting at a round table. In the center of the table is a bowl of rice. Between each pair of philosophers is a single chopstick. A philosopher is in one of the three states: thinking, hungry, or eating. At various times, a thinking philosopher gets hungry. A hungry philosopher attempts to pick up one of the adjacent chopsticks, then the other (not both at the same time). If the philosopher is able to obtain the pair of chopsticks (they are not already in use), then the philosopher eats for a period of time. After eating, the philosopher puts the chopsticks down and returns to thinking. Now, outline a solution using semaphores to solve "The Dining-Philosophers Problem"**

The dining philosopher's problem:

- ⇨ All Chopstick value initially set to 1.
- ⇨ Wait eat korabe
- ⇨ Signal value increase korbe

- The structure of Philosopher *i*:

```
do {
    wait (chopstick[i] );
      wait (chopStick[ (i + 1) % 5] );

        // eat

      signal (chopstick[i] );
      signal (chopstick[ (i + 1) % 5] );

        // think

} while (TRUE);
```

Q.3(c) How do clustered systems differ from multiprocessor systems? Including the initial parent process, how many processes are created by the program shown in **Figure 3(c)**? 5

```
#include <stdio.h>
#include <unistd.h>
int main()
{
int i;
for (i=0; i<3; i++)
fork();
return 0;
}
```

Figure 3(c): How many processes are created?

How do clustered systems differ from multiprocessor systems? Including the initial parent process, how many processes are created by the program shown in Figure 3(c)?

Answer:

1. **Difference between clustered and multiprocessor systems:**
   - **Clustered systems** consist of multiple independent computers (nodes) connected over a network. Each node runs its own operating system and works together to perform tasks, often for high-availability services or parallel processing.
   - **Multiprocessor systems**, on the other hand, consist of a single machine with multiple CPUs (processors) sharing the same memory, operating system, and resources. These processors work together to execute processes in parallel within a single system.
2. **Number of processes created by the program in Figure 3(c):** The given program creates processes using fork() within a loop that runs 3 times (i < 3).
   - Initially, there is 1 parent process.
   - On the first iteration, fork() is called, which creates a new child process. Now, there are 2 processes.
   - On the second iteration, each of the two processes (parent and child) calls fork(), resulting in 4 processes.
   - On the third iteration, each of the four processes calls fork(), leading to a total of 8 processes.

Therefore, the total number of processes created, including the parent process, is **8**.

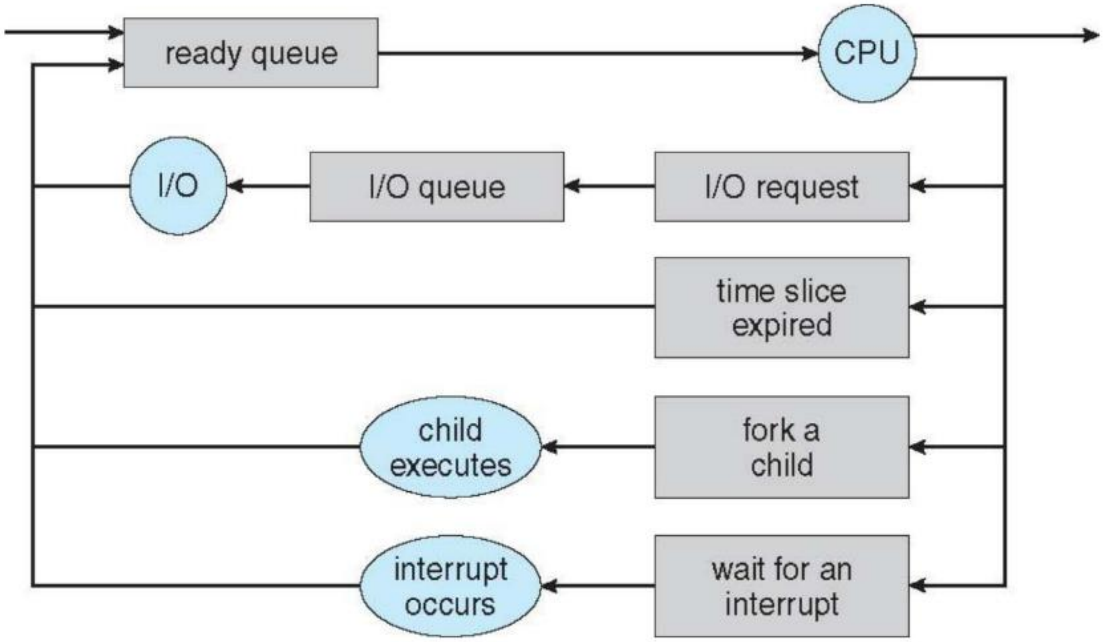
3(d) Answer the following questions:

i) **Explicate the querying diagram representation of process Scheduling.**

ii) **Contrast between the following:**

- **Starvation**
- **Convey Effect**

- **Queueing diagram** represents queues, resources, flows

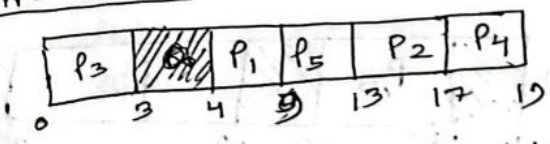


ii)

Convoy effect: It processes with higher burst time arrived before the processes with smaller burst time, then, smaller processes have to wait for a long time for longer processes to release the CPU.

Process ID	Arrival	Burst	T.A.T	W.T.
P_1	4	5	5	0
P_2	6	4	11	7
P_3	0	3	3	0
P_4	6	2	13	11
P_5	5	4	8	4

Gantt chart:

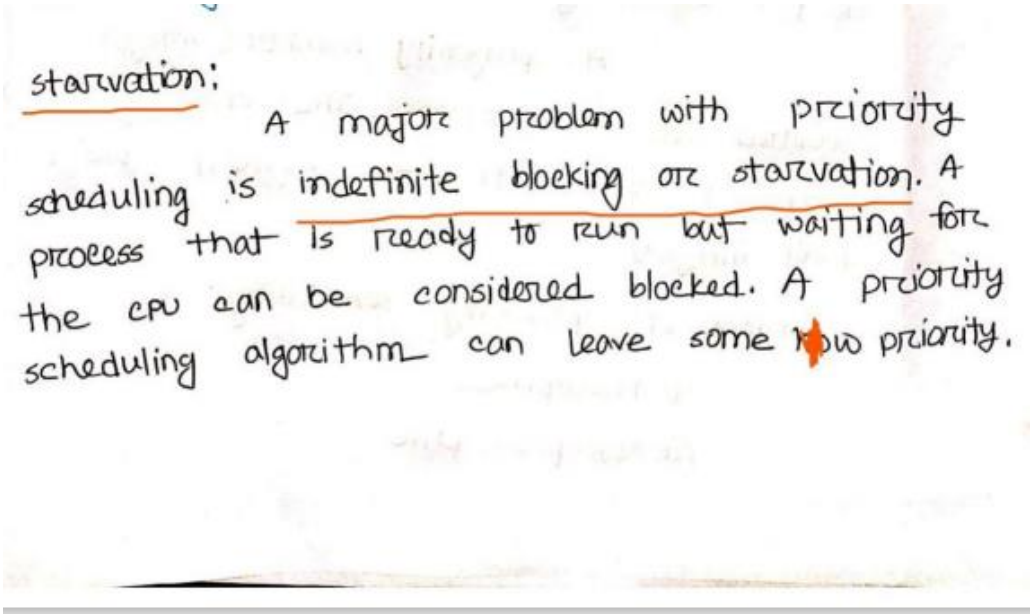


Turn-around time = Completion time - Arrival time

Waiting time = Turn around time - Burst time

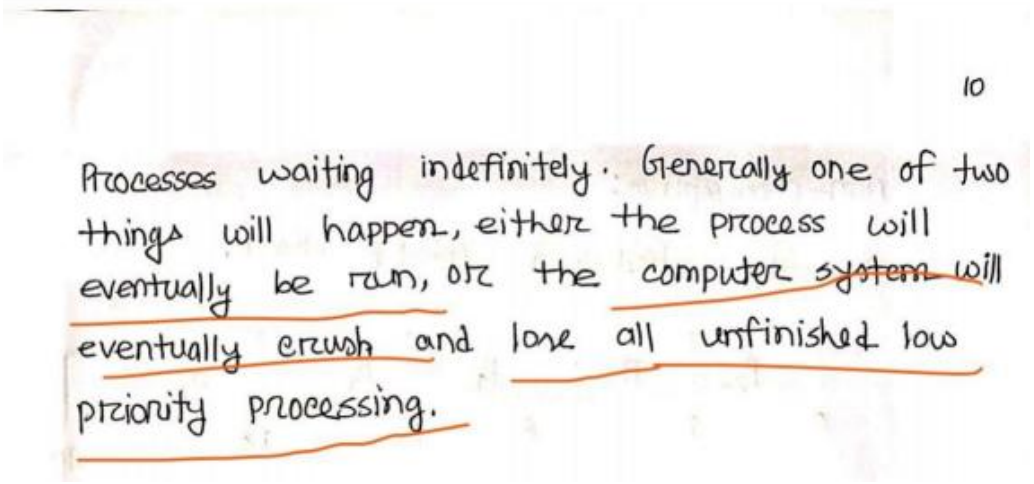
Average Turn around time = $\frac{5+11+3+13+8}{5}$ = 8 units

Waiting time = 0+7+0+11+4 = 4.4 units



starvation:

A major problem with priority scheduling is indefinite blocking or starvation. A process that is ready to run but waiting for the CPU can be considered blocked. A priority scheduling algorithm can leave some low priority.



10

Processes waiting indefinitely. Generally one of two things will happen, either the process will eventually be run, or the computer system will eventually crush and lose all unfinished low priority processing.

Process	Priority	Burst Time (ms)
P1	1	5
P2	2	10
P3	3	15

Starvation of Process P3:

- **P3** has the lowest priority, and if higher-priority processes keep arriving in the system (like P1), P3 will keep waiting indefinitely and will never get the CPU time it needs. This is a classic case of **starvation** for process P3.

4(a) Brief explain the structure of Hashed page table .

Hashed Page Tables:

used in systems with address spaces larger than 32 bits.

Working:

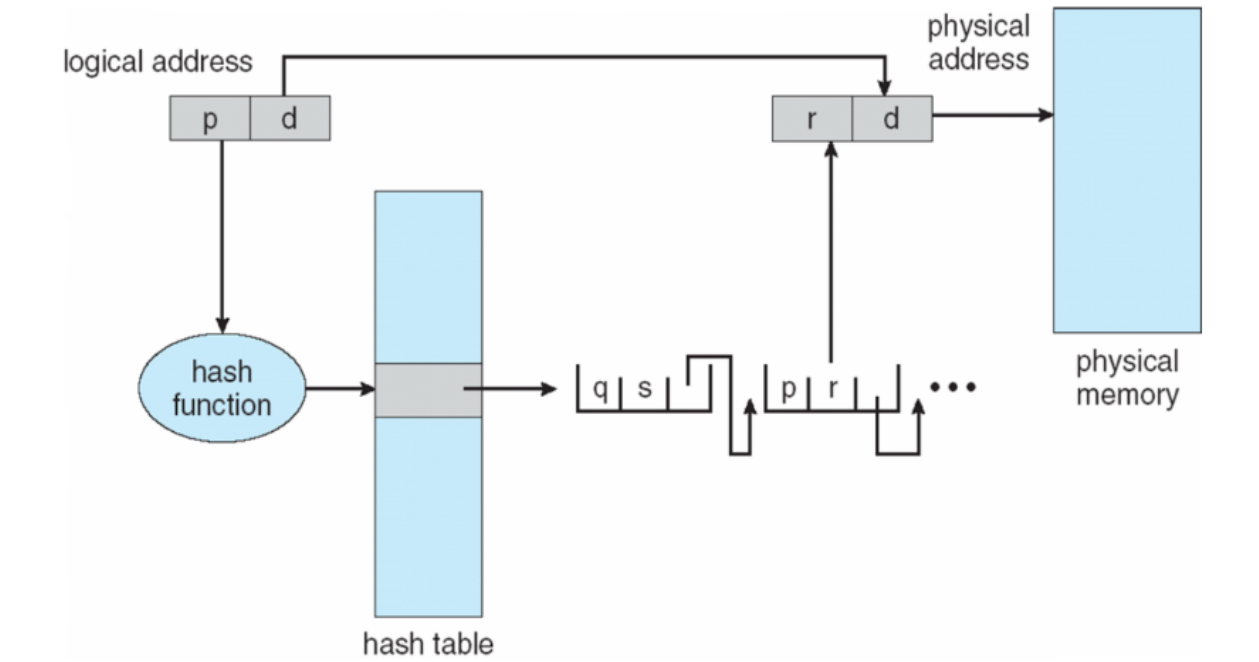
- virtual page number is hashed into a page table.
- hashing function converts the virtual page number into an index
- Each index in the page table contains a chain of elements.
- These elements hash to the same location.

Each element in the chain contains:

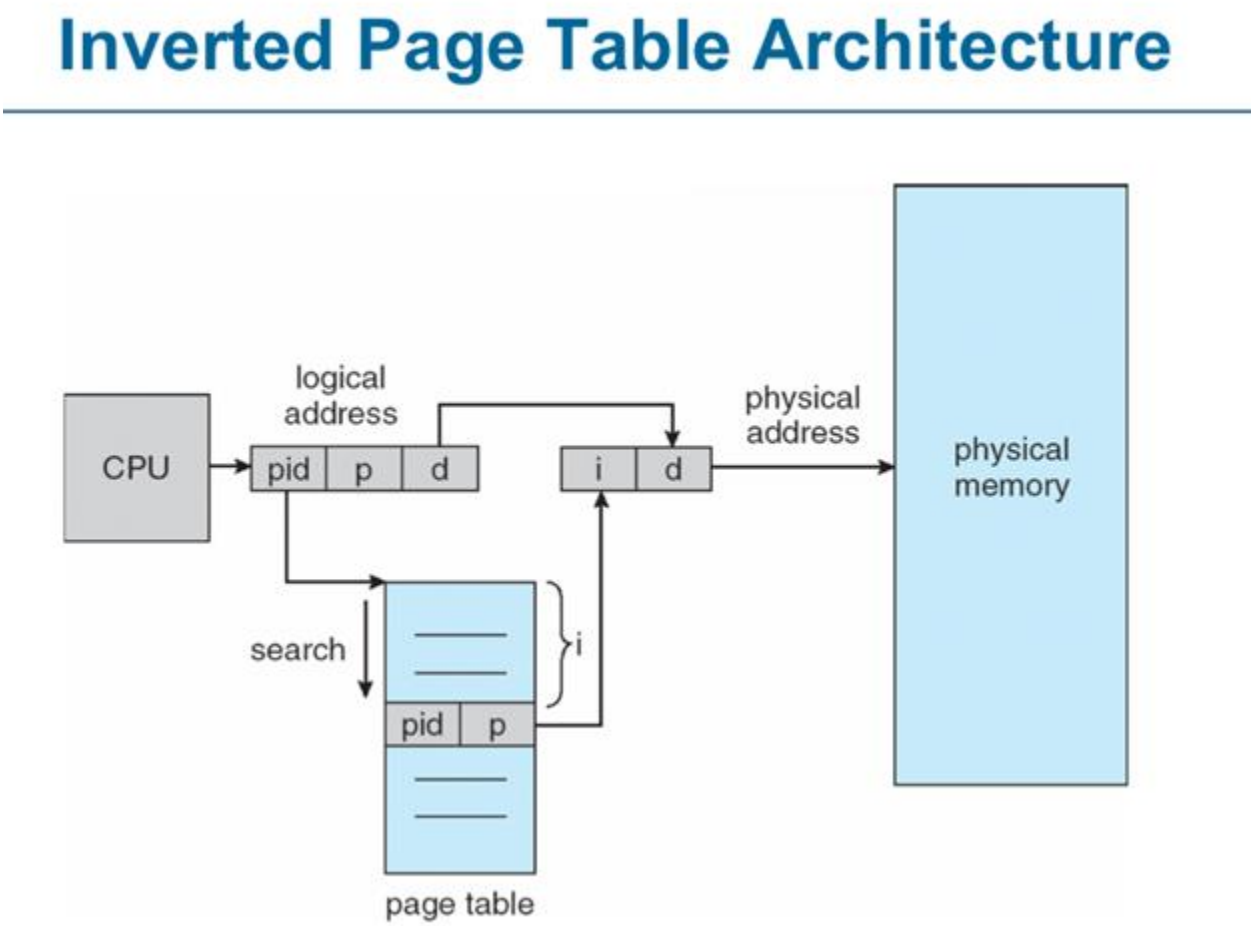
- The virtual page number.
- The corresponding physical frame number.
- A pointer to the next element in the chain.

Search Process:

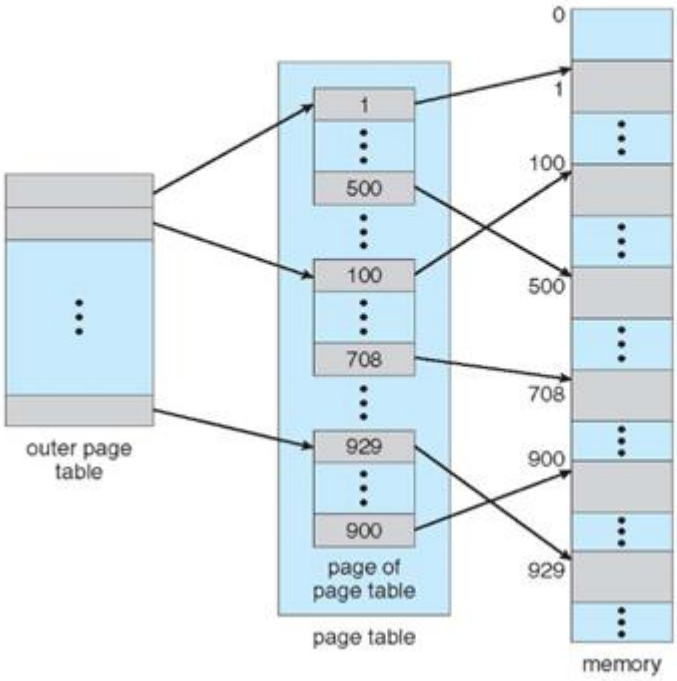
- virtual page number searches through the chain.
- Once a match is found, the corresponding physical frame number is extracted.



4(b) What is an inverted page table? How does it compare to a two-level page table?



Two-Level Page-Table Scheme



Comparison: Inverted Page Table vs. Two-Level Page Table

Feature	Inverted Page Table	Two-Level Page Table
Size of Table	Fixed size, dependent on the number of physical page frames	Variable size, depends on virtual address space
Number of Entries	One per physical page frame	One per virtual page
Memory Efficiency	More efficient for systems with large virtual address spaces	Efficient for sparse virtual address spaces
Lookup Time	Slower, unless using a hash table	Faster, but requires two-level lookup
Process Information	Includes process ID to differentiate between processes	Each process has its own page table
Complexity	More complex due to the need for searching	Easier to implement
Use Case	Suitable for systems with large physical memory and many processes	Suitable for systems with smaller virtual address space

Q.4(c) Consider a memory system which is allocated as specified in Figure 4(c) before 15

Used	Hole	Used	Hole	Used	Hole	Used	Hole	Used	Hole
10 K	10 K	20 K	30 K	10 K	5 K	30 K	20 K	30 K	10 K

Figure 4(c): Variable partition memory allocation

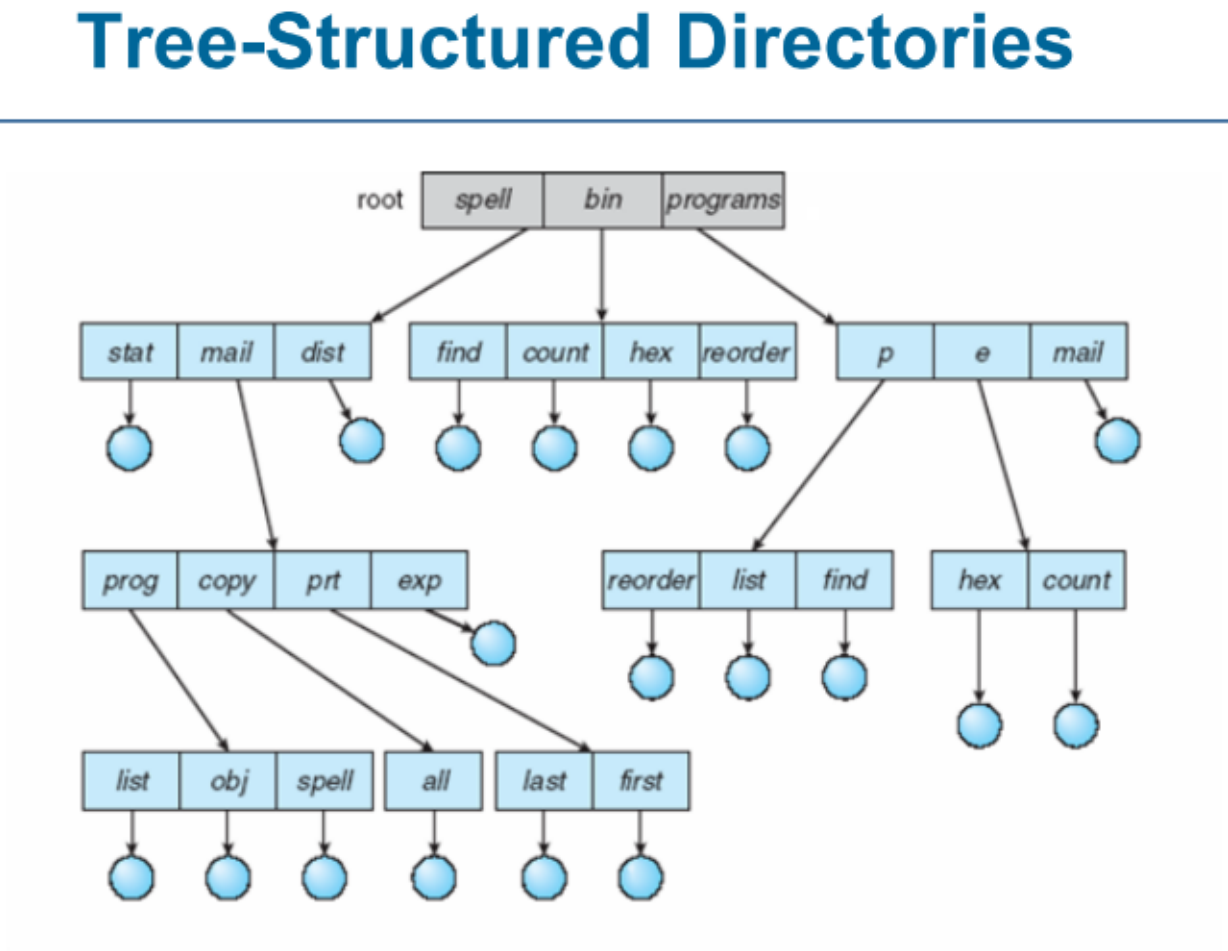
additional requests for 20 K, 15 K, 5 K, 25 K, and 3 K (in order) are received. At what starting address will each of the additional requests be allocated by using

(i) First-Fit Algorithm,

(ii) Best-Fit Algorithm,

(iii) Worst-Fit Algorithm.

4(d) State the problems of tree-structured directory structure .Briefly discuss how this problem can be solved by acyclic-graph directory structure.



Problems of Tree-Structured Directory Structure:

1. Redundant Data:
2. Limited Sharing:
3. Difficulties in Navigation:
4. File Naming Conflicts:
5. Complexity of File Management:

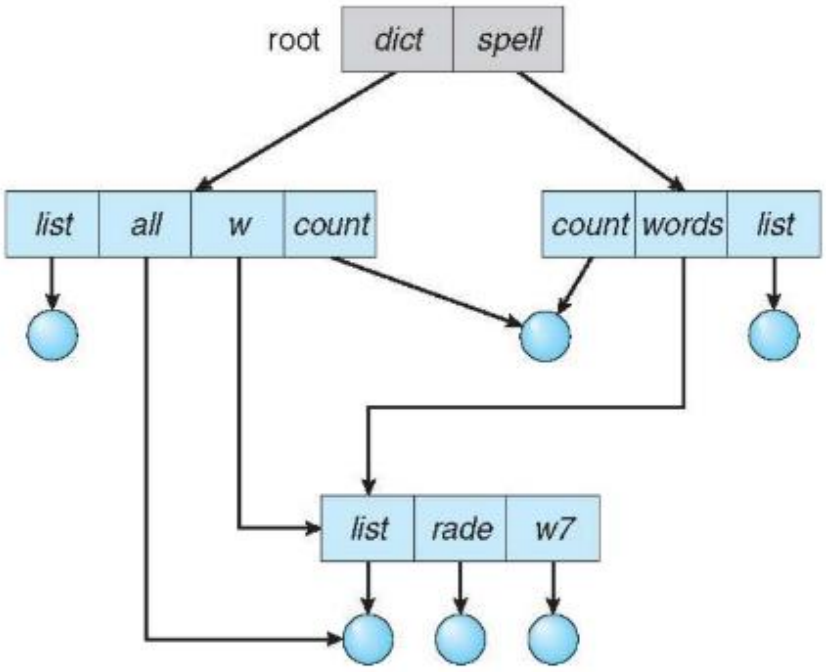
Solution through Acyclic-Graph Directory Structure:

An **acyclic-graph directory structure** addresses the limitations of tree-structured directories by allowing files to have multiple links or references. Here's how it solves the problems:

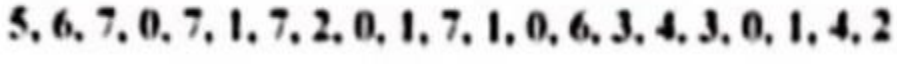
1. Support for Multiple Links:
2. Improved Sharing:
3. Enhanced Navigation:
4. No Naming Conflicts:
5. Simplified File Management:

Acyclic-Graph Directories

- Have shared subdirectories and files
- Example



5(a) Consider the following page reference string:



How many page faults would occur for the following page replacement algorithms utilizing three frames which are empty initially?

i) **FIFO page replacement**
ii) **Optimal page replacement**

5(b) Write short notes on:

- **Relocation Registers.**
- **Compaction.**
- **C-SCAN.**
- **Raid 5.**

1) Relocation Register:

Different methods can be used for mapping logical to physical addresses. A common method involves a **relocation register,**
where a value is added to every address generated by a user process before sending it to memory.

For example, if the base is at 14000,
an attempt by the user to access location 0 is dynamically relocated to location 14000; an access to location 346 is mapped to location 14346.

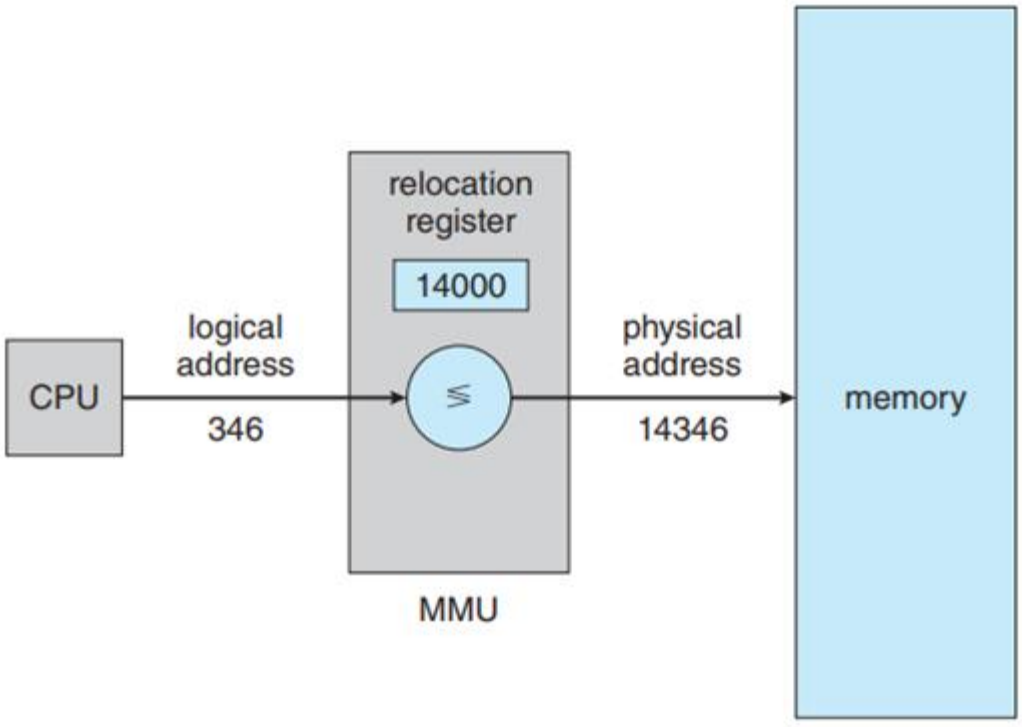


Figure 9.5 Dynamic relocation using a relocation register.

iii) Compaction

Reducing External Fragmentation by Compaction

Compaction is a technique used to reduce external fragmentation by shuffling memory contents to place all free memory together in one large block

Before Compaction:

Memory:

[Allocated Block(20MB),

Free Block(15MB),

Allocated Block(20MB),

Free Block(15),

Allocated Block(10MB)]

After Compaction:

Memory:

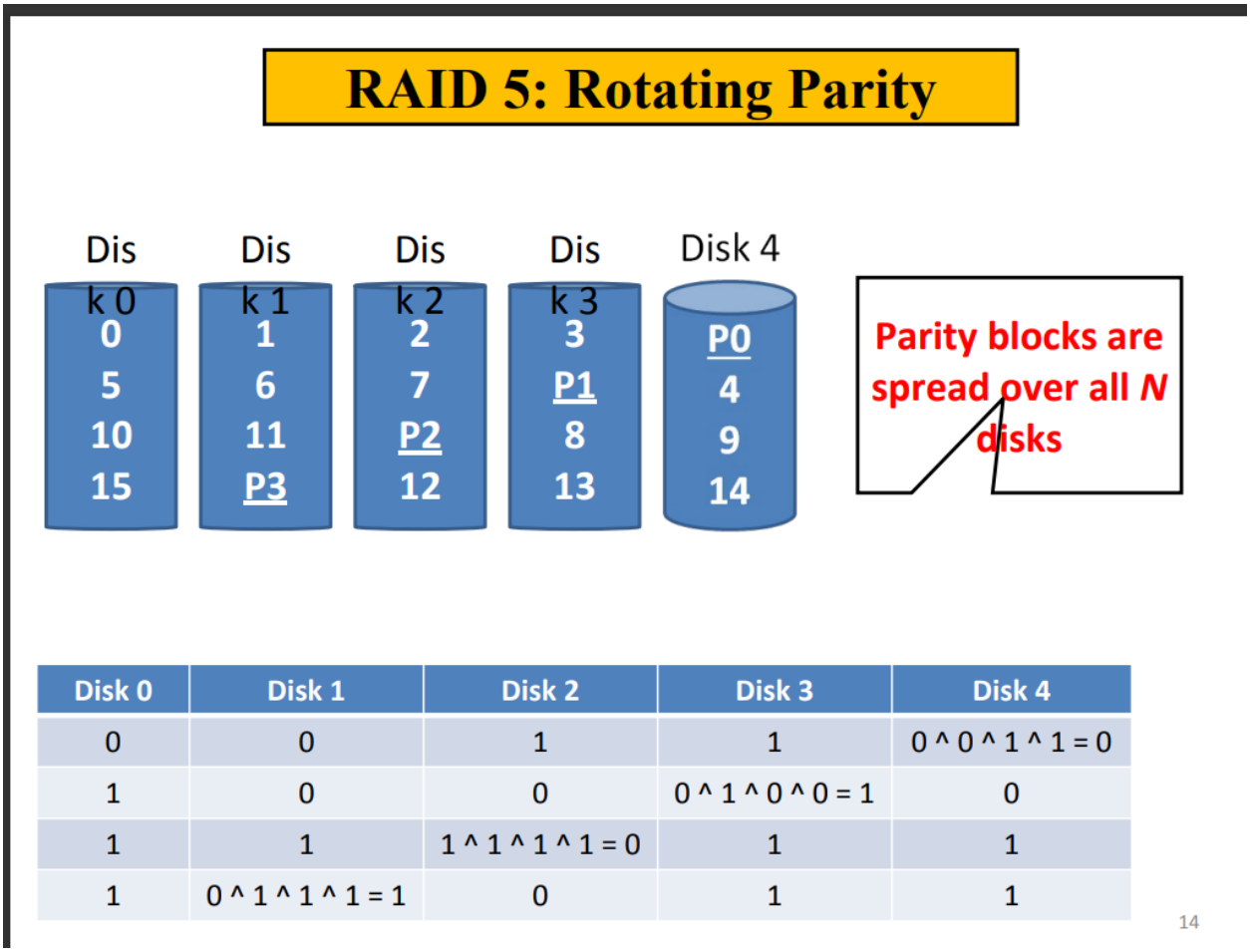
[Allocated Block(20MB),

Allocated Block(20MB),

Allocated Block(10MB),

Free Block(30MB)]

iv) RAID-5



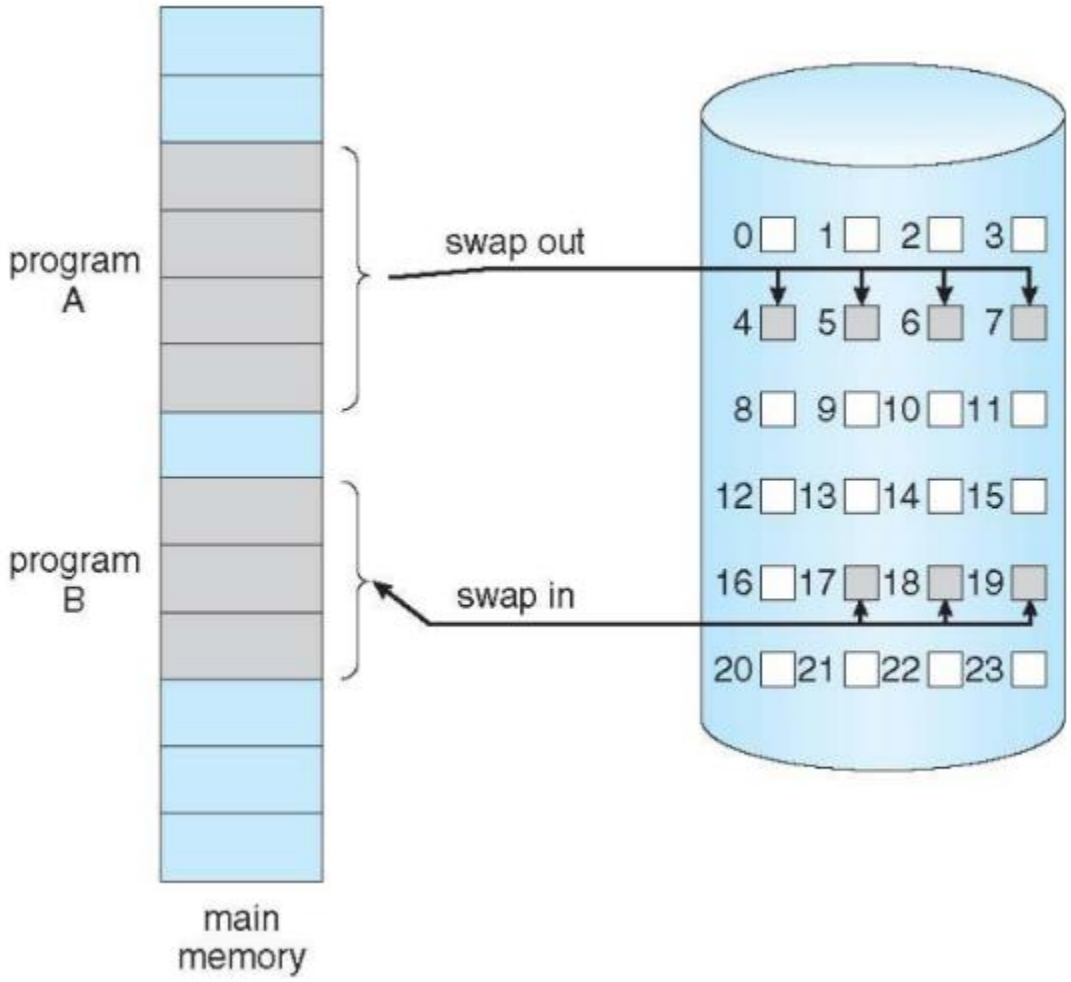
Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	0	1	1	0 ^ 0 ^ 1 ^ 1 = 0
1	0	0	0 ^ 1 ^ 0 ^ 0 = 1	0
1	1	1 ^ 1 ^ 1 ^ 1 = 0	1	1
1	0 ^ 1 ^ 1 ^ 1 = 1	0	1	1

14

5(c) What is demand Paging? How does demand paging affect the performance of the computer system? Explain with examples.

Demand Paging:

Demand paging is a memory management technique in which the operating system loads pages of a process into memory **only when they are needed**. It works similarly to a paging system but with more efficiency in memory usage, as only required pages are loaded.



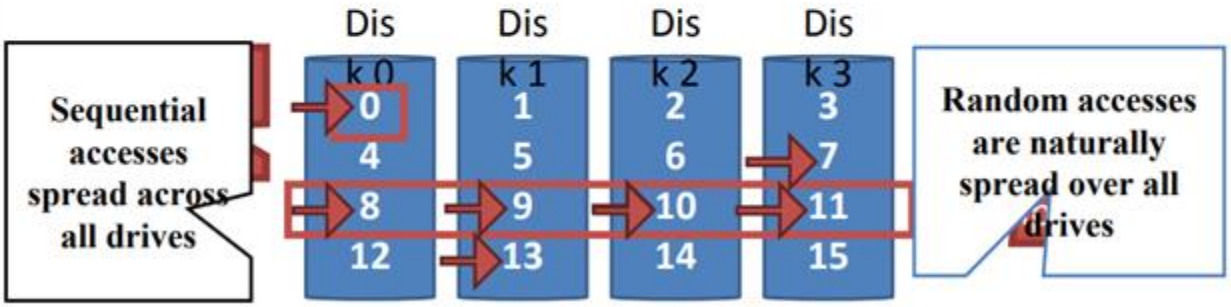
1. **On-Demand Loading:**
    - The operating system **does not load the entire process** into memory at once. Instead, it only loads a page when there is a **reference** (request) to that page.
    - This reduces memory usage and I/O operations, improving efficiency.
2. **Less I/O Operations:**
    - Because only the needed pages are loaded, there are **fewer I/O operations**, resulting in less overhead.
3. **Memory Efficiency:**
    - Since only the required pages are loaded, **less memory** is occupied compared to loading the entire process into memory.
    - This means more processes can fit in memory, leading to **better multitasking**.
4. **Faster Response Time:**
    - The system can start running the process with minimal loading (only a few pages), providing a **faster initial response**.
5. **Similar to Paging with Swapping:**
    - Demand paging is similar to a **paging system with swapping**, where pages are loaded into memory as needed, but the focus is on **minimizing unnecessary swapping and page loading**.

5(d) Explain bit vector implementation of tree-space management?

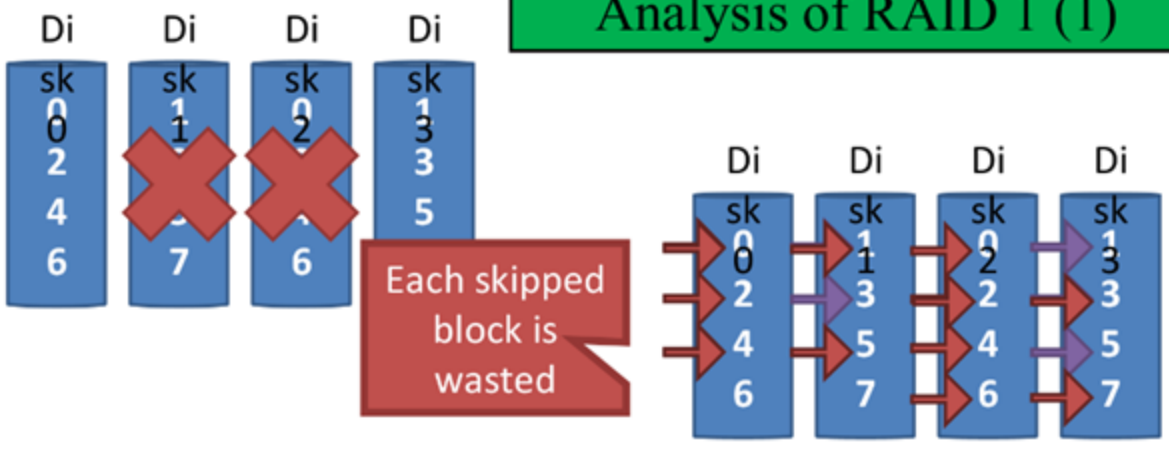
6(a) Compare RAID 0 , RAID 1 and RAID 5 in terms of performance improvement, ability to withstand failure, and space overhead required to implement the particular RAID level.

RAID 0: Striping

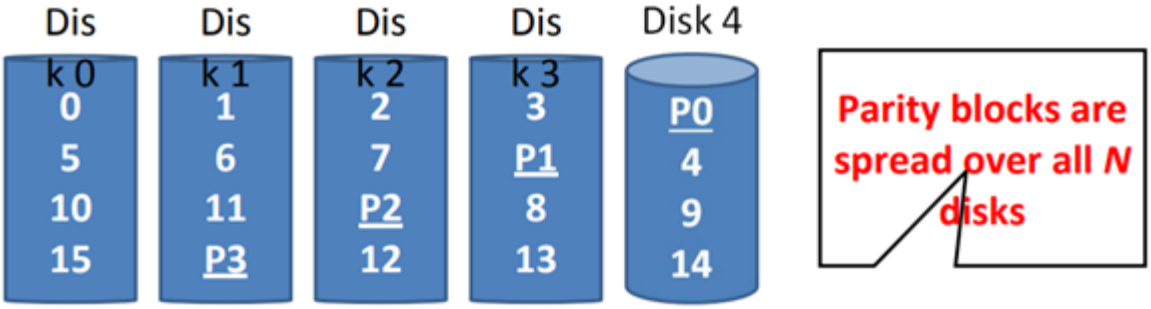
- **Key idea:** present an array of disks as a single large disk
- Maximize parallelism by striping data cross all *N* disks



Analysis of RAID 1 (1)



RAID 5: Rotating Parity



Disk 0	Disk 1	Disk 2	Disk 3	Disk 4
0	0	1	1	0 ^ 0 ^ 1 ^ 1 = 0
1	0	0	0 ^ 1 ^ 0 ^ 0 = 1	0
1	1	1 ^ 1 ^ 1 ^ 1 = 0	1	1
1	0 ^ 1 ^ 1 ^ 1 = 1	0	1	1

6(b) Contrast between protection and security. Write short notes on the following:

- **Denial of Service(DoS)**
- **Trap Door**
- **Trojan Horse**
- **Logic Bom**

Q.6(c) On a disk with 1,000 cylinders, numbered 0 to 999, compute the number of tracks the disk arm must move to satisfy all the requests in the disk queue. Assume that the last request serviced was at track 756 and the head is moving towards track 0. The queue in FIFO order contains requests for the following tracks: 18

450, 811, 348, 153, 968, 407, 500, 371.

Perform the computation for the following disk scheduling algorithms:

i) FIFO

ii) SSTF

iii) SCAN

iv) LOOK

v) C-SCAN, and

vi) C-LOOK.